Unit 2 – Selection and Iteration Name: **KEY**
HW 3

1) Complete the following truth table for the boolean expressions.

| A | B | A&&B | (A&&B)\|\|B | A\|\|B | A&&(A\|\|B) | !A&&B | A\|\|!(A&&B) |
|---|---|---|---|---|---|---|---|
| T | T | T | T | T | T | F | T |
| T | F | F | F | T | T | F | T |
| F | T | F | T | T | F | T | T |
| F | F | F | F | F | F | F | T |

2) Consider the following code, located inside a main method:

```
Scanner scan = new Scanner(System.in);
System.out.println("Enter an integer: ");
int input = scan.nextInt();
```

You will be writing a code segment to check if the number the user has entered is positive and less than 100. The code segment will print TRUE if it is positive and less than 100, and will print false otherwise.

a) Write a code segment that does the above, but does NOT use boolean expressions.

```
if(input > 0)
{
    if(input < 100)
    {
        System.out.println("TRUE");
    }
    System.out.println("FALSE");
}
else
{
    System.out.println("FALSE"):
}
```

b) Write a code segment that does the above, but DOES use boolean expressions.

```
if(input > 0 && input < 100)
{
    System.out.println("TRUE");
}
else
{
    System.out.println("FALSE");
}
```

3) For the following expressions, simplify them using DeMorgan's law.

a) `!(x < 10 || y >= 20)`

`x >= 10 && y < 20`

b) `!(a == b && c != d)`

`a != b || c == d`

c) `!(p <= 5 && q > 15)`

`p > 5 || q <= 15`

d) `!(m == 0 && n < 8) || !(x > 10 && y <= 3)`

`(m == 0 || n >= 8) || (x <= 10 || y > 3)`

4) Suppose you're programming a ticket system for a theme park. The following expression checks if a person is not eligible for a discounted ticket:

```
!(age <= 12 || age >= 65)
```

a) Write the negation of this expression.

`age > 12 && age < 65`

b) In your own words, explain what the negated expression now checks for.

It checks if your age is between 12 and 65, exclusive. If it is, then you are not eligible for a discount.

c) Suppose you want to add a condition that only residents of the state are eligible for the discount. Write an expression that checks if the person should not receive a discount.

You could add a boolean variable called `isResident` and write an expression like this:

`age >12 && age < 65 || !isResident`

This will check if the age is between 12 and 65 or if they are not a resident. If one of these conditions is true, then they do not receive the discounted ticket.

5) A student tried to negate the following expression:

```
!(a < b && b < c)
```

The student's answer was:

```
a > b || b > c
```

Explain the error the student made in your own words.

The student did not include the situation where a is equal to b or b is equal to c. Because this situation was not included in the original expression, we need to include it in the answer:

`a >= b || b >= c`

**D** 7) Consider the following method, `between`, which is intended to return `true` if x is between `lowerBound` and `upperBound`, inclusive, and `false` otherwise.

```
public boolean between(int x, int lowerBound, int upperBound)
{
     /* missing code */
}
```

Which of the following can be used to replace `/* missing code */` so that `between` will work as intended?

(A) `return (x <= lowerBound) && (x >= upperBound);`
(B) `return (x <= lowerBound) || (x >= upperBound);`
(C) `return (x >= lowerBound) || (x <= upperBound);`
(D) `return (x >= lowerBound) && (x <= upperBound);`

**D** 8) What is the output from following code segment? Assume a `Monster` class exists and that it's `equals` method works as intended.

```
Monster frankie = new Monster(4, 5);
Monster goblin = new Monster(4,5);

if (frankie == goblin)
{
     System.out.print("equal ");
}
else
{
     System.out.print("not equal ");
     if(frankie.equals(goblin))
     {
          System.out.print("equal");
     }
}
```

this is false since they do not point to the same location in memory

an equals method that works as intended means that it will compare the instance variables and return true if they are equivalent values.

(A) not equal equal
(B) not equal
(C) equal not equal
(D) equal not equal equal